CS60075 NLP Assignment II — Project 6

A Text Processing Pipeline for Code-Mixed, Real-World Text
SemEval-2020 Task 9 (SentiMix Hinglish), Partition B, with the shared Dakshina lexicon

Group 17

Yashvardhan Jain (23CS10080)
Nishant (23EE10044)
Devubarki Pushyamithra (24CS10092)

Indian Institute of Technology Kharagpur

November 2026

Abstract

We build three Hindi–English sentiment classifiers on Partition B of SentiMix (4,928 train / 616 validation / 616 test tweets). Task 1, a classical pipeline (character n-gram language ID, Dakshina spelling normalization, TF-IDF + Random Forest), reaches **0.564** test macro-F1. Task 2, MuRIL fine-tuned on raw tweets, reaches **0.603**; applying Dakshina normalization to its test input *lowers* this to 0.592. Task 3 adds a 16-dimensional per-token language-tag embedding to MuRIL and trains on normalized text, reaching **0.596**, which is statistically indistinguishable from Task 2 (McNemar $p = 0.85$). The main reason normalization does not help is that the context-free dictionary changes 8.6% of test tokens and some changes alter meaning (e.g. `baad` "after" → `bad`, `har` "every" → `haar` "defeat").

1 Introduction

Code-mixed tweets such as *"yeh movie bahut acchi thi, must watch!"* mix Romanized Hindi and English in one sentence. Romanized Hindi has no standard spelling (*bahut*/*bhut*/*bohot*), and the same string can be a word in either language (*main*, *par*). We ask three questions, one per task:

1. How well does an explicit classical pipeline (language ID → normalization → classifier) do?
2. How much of that hand-engineering does MuRIL, a model pretrained on Indian languages and transliterated text, make unnecessary, and does normalizing its input at test time help?
3. Are language-ID tags and normalization complementary to MuRIL when fed into it directly?

2 Data and Preprocessing

SentiMix Partition B. We parse `Train.txt`, `Validation.txt` and `Test.txt` from the `singhnivedita/SemEval2020-Task9` repository, taking test labels from `Ty.txt`. A line counts as a tweet header only if it is exactly `meta`, a numeric id and (outside the test file) a sentiment label, so tokens like *metaphysist* cannot start a new tweet. Removing the 2,650 ids duplicated across train and validation leaves 18,480 tweets. We shuffle this pool once with seed 42, take the middle third as Partition B (6,160 tweets), shuffle it again with seed 42 and split it 80/10/10. The test split has 160 negative, 241 neutral and 215 positive tweets.

Noise handling. Links are split across tokens in the corpus (`https // t . co / oxf8tr3bly`). We remove every token from `http` through the short-link id, including truncated links (`https // t . c ...`) and an id left dangling after a truncated link. For mentions, we remove the `@` token (alone or attached to punctuation, as in `.@`) and the username token after it. Hashtags and emojis are kept as sentiment cues. One limitation remains: usernames containing _ are split into several tokens, so only their first piece is removed (e.g. _ `Bhandari` survives).

Dakshina dictionary. From the Hindi word-pair lexicons (`hi.translit.sampled.{train, dev,test}.tsv`, all splits combined) we map every attested romanization of a Devanagari word to that word's most frequent romanization, giving 49,090 mappings (e.g. `kuch` → `kuchh`, `nahi` → `nahin`). A romanization attested for several Devanagari words is mapped to the canonical form of whichever word is processed last. Section 5 shows that this causes meaning-changing rewrites.

3 Methods

Task 1: classical pipeline.

1. **Language ID:** character 1–4-gram TF-IDF features and logistic regression (L2, $C = 1.0$), trained per token on the gold {`Hin`, `Eng`, `O`} tags of the 96,516 training tokens.
2. **Normalization:** every token the tagger does not label `Eng` is looked up case-insensitively in the Dakshina dictionary and replaced by its canonical form. All other tokens keep their original casing.
3. **Classifier:** word and character 1–3-gram TF-IDF features over the normalized text, fed to a Random Forest with 300 trees, maximum depth 20 and at least 5 samples per leaf.

Task 2: MuRIL. We fine-tune `google/muril-base-cased` with a 3-way classification head on the raw (non-normalized) training tweets, using AdamW with learning rate 2×10^{-5}, batch size 16, 4 epochs and maximum length 128. The same checkpoint is then evaluated twice on the test split: once on raw text, and once on text normalized by Task 1's tagger and dictionary, with no retraining.

Task 3: LID-augmented MuRIL. We implement the per-token option of the specification. Each subword takes the Task 1 predicted tag of its word (special and padding tokens get a separate `PAD` tag), which is embedded in $\mathbb{R}^{16}$ and concatenated to MuRIL's word embedding $\mathbf{e}_j$ before the transformer layers. The 784-dimensional vector is projected back to 768 dimensions:

$$\tilde{\mathbf{e}}_j = [\,\mathbf{I}_{768} \mid \mathbf{W}_{\text{lid}}\,] \begin{bmatrix} \mathbf{e}_j \\ \mathbf{z}_j \end{bmatrix} = \mathbf{e}_j + \mathbf{W}_{\text{lid}}\,\mathbf{z}_j, \tag{1}$$

where $\mathbf{W}_{\text{lid}} \in \mathbb{R}^{768\times16}$ starts at zero, so training starts from exactly pretrained MuRIL. Position embeddings and LayerNorm are applied afterwards as usual. The input is the Dakshina-normalized text, and the hyperparameters are those of Task 2.

We first trained a fully learned 784×768 projection initialized to $[\,\mathbf{I} \mid \mathbf{0}\,]$. That model collapsed to predicting `neutral` for every tweet (test macro-F1 0.188). Checks on Colab ruled out the library: MuRIL gives identical outputs for token ids and pre-computed embeddings, and no weights were missing. Our explanation is that AdamW moves each of the projection's ∼600k weights by roughly the learning rate per step regardless of gradient size. Summed over 768 inputs, that drift soon outweighs MuRIL's small word embeddings (standard deviation 0.07) and destroys the pretrained input space. Fixing the word block at the identity removes this failure.

Deviations from the specification. Task 1 adds `class_weight="balanced"` and caps each TF-IDF vocabulary at 25,000 features. Tasks 2 and 3 use AdamW weight decay 0.01, linear warmup over 10% of steps, gradient clipping at 1.0, and keep the epoch with the best validation macro-F1. All runs use one seed (42) on Google Colab with a T4 GPU, Python 3.13, torch 2.11, transformers 5.22 and scikit-learn 1.6.1.

4 Results

Table 1: Sentiment results on Partition B. The test split has 616 tweets. "Norm" is the Dakshina normalization; RF is the Random Forest.

Configuration	Val macro-F1	Test macro-F1	Test accuracy
Task 1: LID + norm + TF-IDF RF	0.5589	0.5641	0.5698
Task 2: MuRIL, raw input	**0.6691**	**0.6025**	**0.6055**
Task 2 check: norm at test only	—	0.5915	0.5942
Task 3: MuRIL + LID emb. + norm	0.6377	0.5957	0.6006

Table 2: Per-class test F1.

Model	negative (160)	neutral (241)	positive (215)
Task 1	0.609	0.425	0.658
Task 2	0.623	**0.477**	0.708
Task 3	**0.626**	0.441	**0.720**

The Task 1 language-ID tagger reaches 0.867 token accuracy and 0.885 macro-F1 on the 12,120 test tokens.

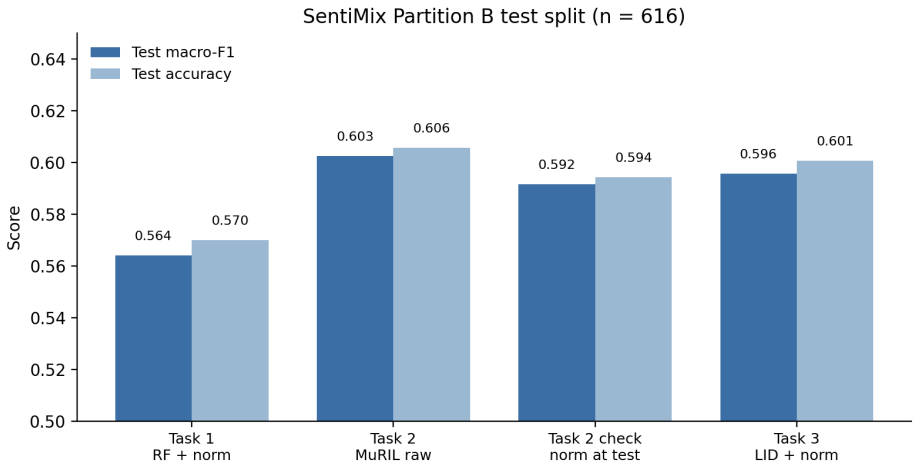


Figure 1: Test macro-F1 and accuracy for each configuration.

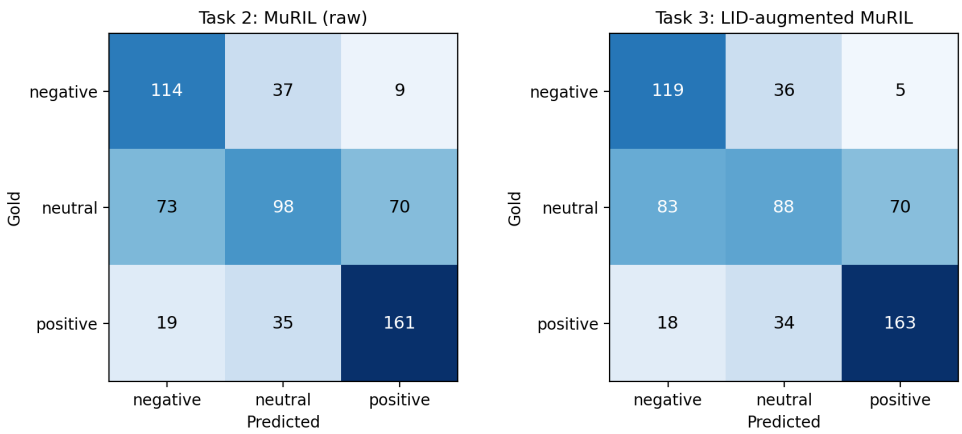


Figure 2: Test confusion matrices for Task 2 (left) and Task 3 (right).

5 Analysis

Task 1 vs. Task 2. MuRIL improves on the Random Forest by 3.8 test macro-F1 points, with gains in every class. A bag of n-grams cannot represent word order or negation, while MuRIL's pretraining covers transliterated Hindi. All three models find `neutral` hardest, confusing it with `negative` and `positive` about equally often (Figure 2). Task 1 also over-predicts `negative` (recall 0.78, precision 0.50).

Normalization hurts the raw-trained model. Normalizing only the test input costs Task 2 1.1 points. The dictionary changes the spelling of 1,042 of the 12,120 test tokens (8.6%), in 458 of the 616 tweets. Many of these are intended variant fixes (`nahi` → `nahin`, `kuch` → `kuchh`). Others change meaning, because the dictionary ignores context and resolves romanizations shared by several Hindi words arbitrarily:

Table 3: Frequent meaning-changing rewrites in the test split.

Rewrite	Count	Effect
`ji` → `ge`	57	honorific becomes a meaningless string
`he` → `hey`	25	Hindi "is" becomes the English interjection
`par` → `paar`	20	"on/but" becomes "across"
`har` → `haar`	15	"every" becomes "defeat"
`main` → `man`	15	pronoun "I" becomes "mind"
`baad` → `bad`	10	"after" becomes the English word "bad"
`kam` → `kaam`	9	"less" becomes "work"

Even correct rewrites move the text away from the raw spellings MuRIL was fine-tuned on. So Task 2's answer to the specification's question is that this normalization *hurts* a model fine-tuned on raw text.

Why Task 3 does not improve on Task 2. Task 3 is 0.7 points below Task 2. Of the 616 test tweets, 58 are classified correctly only by Task 2 and 55 only by Task 3, so the difference is not significant (exact McNemar test, $p = 0.85$). We see three reasons:

1. **Normalization cancels any gain.** Task 3 trains and tests on normalized text, which costs Task 2 about 1.1 points, so the language-tag embedding may be helping and still be masked.

Separating the two effects needs one more run (LID embedding on raw text), which we have not done.

2. **The tags are noisy and redundant.** The tagger is wrong on about 13% of tokens, and on the training split it tags data it was trained on, so training tags are cleaner than test tags. MuRIL's subwords already carry strong language cues, so a 3-way tag adds little.
3. **Single small-data run.** With 4,928 training tweets and one seed, differences below about 1–2 points are within run-to-run variation. Task 3's validation score also dipped at epoch 2 ($0.517 \rightarrow 0.455$) before recovering.

6 Paper Reproduction Comparison

The MuRIL paper [2] does not evaluate on SentiMix. It reports zero-shot results on XTREME tasks (e.g. named-entity recognition, POS tagging and natural language inference), where MuRIL beats mBERT, most clearly on transliterated test sets. There is therefore no published MuRIL SentiMix number to match. The nearest reference is the SemEval-2020 Task 9 shared task [1], where the best Hinglish system reached 75.0% F1, and BERT-style models and ensembles were the most successful approaches. Our 0.603 is well below that, as expected:

- **Data:** we train on 4,928 tweets, about a third of the full training set, because the corpus is split between three groups.
- **Budget:** we use one base-size model, 4 epochs and fixed hyperparameters, with no ensembling or external data.
- **Evaluation:** our test split is a random 616-tweet sample of the pooled data, not the official test set, so the numbers are not directly comparable.

7 Qualitative Analysis

Each example shows the cleaned tweet, the gold label and the predictions. "Norm" lists the dictionary rewrites.

Task 3 right, Task 2 wrong.

- `_ Bhandari what about Jhootha and Don Modi` — gold **negative**; Task 2 neutral, Task 3 negative. Norm: `Jhootha` → `jhutha` ("liar"). The canonical spelling of the insult is a more common form, which plausibly helped Task 3.
- `Finally we didn ' t change cowkidaar jaisa bhi imandaar hey` — gold **positive**; Task 2 neutral, Task 3 positive. No rewrites, so the difference comes from the language tags or from training on normalized text, not from normalizing this tweet.

Task 2 right, Task 3 wrong.

- `Kya chal kya raha hai !? [emoji] # jcbkikhudayi` ("what is going on?") — gold **neutral**; Task 2 neutral, Task 3 negative. Norm: `chal` → `chaal`, which means "trick, scheme" and plausibly pushes the prediction towards negative.
- `Mtlb koi bat hi nahi shraddha kapoor smjh bethi khud ko [emoji]` — gold **neutral**; Task 2 neutral, Task 3 negative. Norm: `bat` ("matter") → `bet`, an English word with a different meaning.

Both wrong.

- `Love day lage hue he jindagi ke [emoji]` — gold **negative**; both predict positive. The sarcasm is carried by the eye-roll emoji against the literal "love". Norm also rewrote `he` → `hey`.
- `RT Sabka Saath Sabka Vikas Sabka Viswas |` — gold **neutral**; both predict positive. A political slogan quoted without comment reads as positive on the surface. Telling it apart needs context that neither model has.

8 Discussion and Conclusion

What worked. The character n-gram language-ID tagger is cheap and reasonably accurate (0.867 token accuracy). Fine-tuned MuRIL is clearly the best single component, 3.8 points above the classical pipeline. Fixing the word block of the Task 3 projection at the identity made LID augmentation trainable, after the fully learned projection collapsed.

What did not. Dakshina normalization did not help either neural model. A context-free dictionary built by keeping the most frequent romanization rewrites common Hindi function words into other words, so it adds noise, while MuRIL already handles spelling variation from pretraining. Adding language tags gave no significant gain on top of normalized input.

Conclusion. On this partition, MuRIL absorbs most of what the classical pipeline provides. Explicit normalization should be applied only where it is reliable. The clearest next steps are: (i) a Task 3 run on raw text, to isolate the effect of the tags; (ii) a dictionary that skips romanizations shared by several Hindi words; and (iii) several seeds per model, to put error bars on differences of this size.

References

[1] P. Patwa, G. Aguilar, S. Kar, S. Pandey, S. PYKL, B. Gambäck, T. Chakraborty, T. Solorio and A. Das. 2020. SemEval-2020 Task 9: Overview of Sentiment Analysis of Code-Mixed Tweets. In *Proc. SemEval-2020*, pp. 774–790.

[2] S. Khanuja, D. Bansal, S. Mehtani, S. Khosla, A. Dey, B. Gopalan, D. K. Margam, P. Aggarwal, R. T. Nagipogu, S. Dave, S. Gupta, S. C. B. Gali, V. Subramanian and P. Talukdar. 2021. MuRIL: Multilingual Representations for Indian Languages. arXiv:2103.10730.

[3] B. Roark, L. Wolf-Sonkin, C. Kirov, S. J. Mielke, C. Johny, I. Demirsahin and K. Hall. 2020. Processing South Asian Languages Written in the Latin Script: the Dakshina Dataset. In *Proc. LREC 2020*, pp. 2413–2423.